

Chain-of-Thought and Reasoning Complexity

English seminar study notes

These notes are an English companion to the seminar handout. The emphasis is on mathematical derivations that can be followed line by line, on separating theorem statements from interpretation, and on identifying the assumptions under which each claim is valid.

Primary readings

Merrill and Sabharwal, work on CoT computational expressivity.

Bai et al., **Transformers as Statisticians**.

Wu et al., **In-Context Deep Learning via Transformer Models**.

Contents

1	Notation and four theoretical axes	2
1.1	Chain of Thought as an iterated computation	2
1.2	Four questions that should be kept distinct	2
2	CoT as a sequential computational resource	3
2.1	One forward pass versus repeated decoding	3
2.2	A computational sandwich	3
2.3	DFA simulation	3
2.4	Do not simplify this to “CoT = P”	3
3	An optimization loop inside ICL: from attention to backprop	4
3.1	One linear-attention layer can implement one GD step	4
3.2	Akyürek et al.: the same family of ideas	4
3.3	Bai et al.: multi-step GD and in-context algorithm selection	5
3.4	Wu et al.: simulating deep-network backpropagation	5
3.5	The important axes are not encoder versus decoder	6
3.6	Connection to Chapter 2 and to faithfulness	6
4	Learning autoregressive CoT: sample complexity	7
4.1	End-to-end versus CoT supervision	7
4.2	Hanneke–Mehalel–Moran: removing T from sample count	7
5	How informative is the intermediate trace?	8
5.1	CoT information	8
6	Reasoning-token lower bounds: BAPO	9
6.1	Why a separate lower bound is needed	9
6.2	Representative lower bounds	9
6.3	Why this does not contradict sample-complexity gains	9
7	Reasoning length, error accumulation, and faithfulness	10
7.1	More tokens are not automatically better	10
7.2	Faithfulness is an orthogonal axis	10
8	Putting the four axes in one table	11
8.1	Five formulas worth keeping	11
8.2	Common mistakes	11
	Bibliography	12

1

Notation and four theoretical axes

1.1 Chain of Thought as an iterated computation

Let a fixed decoder implement a next-step map f . Given input x , a reasoning trace of length T is

$$z_1 = f(x), \quad z_2 = f(x, z_1), \quad \dots \quad z_T = f(x, z_{1:T-1}).$$

The final answer may be a function of the transcript $z_{1:T}$. This notation makes explicit that the same network is reused repeatedly.

1.2 Four questions that should be kept distinct

1. **Computational depth:** what does autoregressive reasoning add beyond one forward pass?
2. **ICL $\leftrightarrow$ optimization:** can one forward pass itself contain a learning/optimization loop?
3. **Learning signal:** how does observing intermediate traces change sample complexity?
4. **Token lower bounds:** how many inference-time reasoning tokens are necessary for some tasks?

The first two concern computation; the third concerns statistical supervision; the fourth concerns an inference resource.

How to say it in the presentation This week is easiest to understand if “reasoning tokens” are treated as one resource among several. Network depth can simulate an optimizer inside one pass, while autoregressive decoding adds another sequential resource outside the pass.

2

CoT as a sequential computational resource

2.1 One forward pass versus repeated decoding

A fixed-depth Transformer forward pass performs a bounded number of parallel transformation rounds. Autoregressive CoT feeds the output back as input, so T generated tokens create T additional sequential rounds. It is useful to distinguish

$$D_{\text{inner}} = \text{architectural depth inside one forward pass}$$

from

$$T_{\text{decode}} = \text{number of autoregressive reasoning steps.}$$

2.2 A computational sandwich

Under the formal decoder model used in CoT expressivity theory, one obtains relations of the schematic form

$$\boxed{\text{TIME}(t(n)) \subseteq \text{CoT}(t(n))}$$

and an upper simulation by limited workspace such as

$$\boxed{\text{CoT}(t(n)) \subseteq \text{SPACE}(t(n) + \log n)}$$

with the exact encoding and precision assumptions supplied by the theorem. The left inclusion interprets one reasoning token as one simulated algorithmic step; the right stores the transcript and recomputes bounded-depth forward passes.

2.3 DFA simulation

For a DFA transition $q_i = \delta(q_{i-1}, x_i)$, a linear-length scratchpad can simply emit q_i after reading each symbol. CoT therefore externalizes the recurrent state that a fixed one-pass computation may not have enough sequential depth to maintain.

2.4 Do not simplify this to “CoT = P”

A complexity characterization depends on time bounds, numerical precision, uniformity, and the exact Transformer abstraction. Expressivity also does not imply that SGD learns the simulation or that the generated trace is human-interpretable.

How to say it in the presentation CoT adds sequential time by repeatedly reusing the same network. That is different from making a single forward pass deeper, and different again from giving intermediate labels during training.

3

An optimization loop inside ICL: from attention to backprop

3.1 One linear-attention layer can implement one GD step

Consider linear regression with current weight matrix W and examples $(x_i, y_i)_{i=1}^N$. One MSE gradient step is

$$\Delta W = -\frac{\eta}{N} \sum_{i=1}^N (Wx_i - y_i)x_i^\top, \quad W^+ = W + \Delta W.$$

For a query x_j ,

$$\Delta W x_j = -\frac{\eta}{N} \sum_i (Wx_i - y_i)(x_i^\top x_j).$$

A linear-attention head computes

$$PVK^\top q = P \sum_i v_i (k_i^\top q).$$

Choose block-structured token embeddings and projections so that

$$k_i^\top q_j = x_i^\top x_j, \quad v_i = \begin{bmatrix} 0 \\ Wx_i - y_i \end{bmatrix}.$$

With the output projection scaled by η/N , the update to the prediction slot is

$$\frac{\eta}{N} \sum_i (Wx_i - y_i)(x_i^\top x_j) = -\Delta W x_j.$$

Hence the post-attention token can encode the prediction of the GD-updated predictor.

From the original paper The exact von Oswald construction is for one-head linear self-attention with a particular regression tokenization. It is an explicit representability result. Their experiments additionally show settings where meta-training finds behavior close to the construction, but the theorem itself does not state that ordinary softmax LLM training must do so.

3.2 Akyürek et al.: the same family of ideas

Akyürek et al. construct Transformer primitives that implement gradient-style updates and regression estimators using a constant number of layers and $O(d)$ hidden state per update in controlled linear-model settings. Stacking the primitive implements multiple inner updates. The shared message is

$$\text{context} \mapsto \text{task-specific optimizer state in activations.}$$

3.3 Bai et al.: multi-step GD and in-context algorithm selection

Let

$$\widehat{L}_N(w) = \frac{1}{N} \sum_{i=1}^N \ell(w^\top x_i, y_i)$$

be a smooth convex empirical risk and

$$w^{t+1} = w^t - \eta \nabla \widehat{L}_N(w^t).$$

Bai et al. construct an attention-only Transformer in which one layer approximates one in-context GD iterate; an $(L + 1)$ -layer model can therefore implement L such iterates with controlled accumulated error under their assumptions.

They also move from a single base algorithm to *algorithm selection*. A post-ICL validation scheme has the conceptual form

$$\widehat{k} = \arg \min_{k \in [K]} \widehat{R}_{\text{val}}(A_k(D_{\text{train}})),$$

while a pre-ICL routing scheme first computes a context statistic and sets

$$\widehat{k} = g(s(D)).$$

Thus the Transformer can be constructed not only to run an inner algorithm but to select which algorithm to run from context.

Caution The central Bai constructions use a normalized ReLU-attention primitive rather than ordinary softmax attention. The precise claim is therefore architectural and constructive, not “every standard Transformer performs gradient descent.”

3.4 Wu et al.: simulating deep-network backpropagation

The next step is much stronger. Let w denote the parameters of an N -layer target network and $L_n(w)$ its empirical risk. The inner optimizer is

$$w^{(\ell+1)} = \text{Proj}_{\mathcal{W}} \left(w^{(\ell)} - \eta \nabla L_n(w^{(\ell)}) \right).$$

Computing one update requires a forward sweep through the target network, a reverse-mode chain-rule sweep, aggregation of sample gradients, and an update/projection. Wu et al. unfold these operations across Transformer depth.

The quantitative theorem has the form

$$L \text{ GD steps on an } N\text{-layer network} \iff (2N + 4)L\text{-layer Transformer.}$$

Each simulated step is an inexact update

$$\widehat{w}^{(\ell)} = \text{Proj}_{\mathcal{W}} \left(\widehat{w}^{(\ell-1)} - \eta \nabla L_n(\widehat{w}^{(\ell-1)}) + \varepsilon^{(\ell-1)} \right), \quad \|\varepsilon^{(\ell-1)}\|_2 \leq \eta \epsilon.$$

The primary construction uses ReLU-style attention and the paper separately studies softmax extensions.

Caution The result with the $(2N + 4)L$ depth appears in Wu et al., arXiv:2411.16549, currently titled *In-Context Deep Learning via Transformer Models*. The formal theorem includes smoothness assumptions on the target activation and its derivative; it should not be read as an unconditional exact theorem for nonsmooth ReLU networks.

3.5 The important axes are not encoder versus decoder

A more informative comparison is

$$\text{linear/ReLU-style attention} \leftrightarrow \text{softmax attention}$$

and

$$\text{one shallow estimator update} \leftrightarrow \text{a full deep forward/backward simulation.}$$

Encoder/decoder masking changes information routing, but the central theoretical question is which optimizer circuit can be embedded into the forward computation.

3.6 Connection to Chapter 2 and to faithfulness

The CoT results in Chapter 2 obtain serial computation by chaining many forward passes. The ICL-backprop results instead spend architectural depth within one forward pass. Both resources may be used together.

There are also three different strengths of claim:

1. **Representability:** parameters exist that compute the same function as the optimizer.
2. **Learnability of the circuit:** outer-loop SGD actually finds such a circuit.
3. **Mechanistic faithfulness:** on a natural task, the model causally uses that optimizer-like mechanism.

The construction theorems prove the first most directly. Evidence for the second or third requires additional arguments.

How to say it in the presentation The ICL-backprop axis runs from “one linear-attention layer exactly implements one regression GD step” to “a $(2N + 4)L$ -layer Transformer simulates L backpropagation steps of an N -layer target network.” The forward pass can therefore contain a small training loop. The essential caveat is that existence of such a circuit is not a proof that SGD always discovers it in real LLMs.

4

Learning autoregressive CoT: sample complexity

4.1 End-to-end versus CoT supervision

Let $\mathcal{F}$ be a base next-token class. Repeated application for T steps defines an end-to-end class $\mathcal{F}^{\text{e2e},T}$. With only final-answer supervision, Joshi et al. obtain bounds of the schematic form

$$\text{VCdim}(\mathcal{F}^{\text{e2e},T}) \leq 6T \text{VCdim}(\mathcal{F})$$

in a binary setting, which leads to realizable PAC sample complexity roughly

$$m_{\text{e2e}} = O\left(\frac{T \text{VCdim}(\mathcal{F}) \log(1/\epsilon) + \log(1/\delta)}{\epsilon}\right).$$

They also construct classes for which linear dependence on T is genuinely necessary up to constants/log factors.

With CoT supervision, one trajectory exposes the T transition constraints

$$f_{\star}(x) = z_1, \quad f_{\star}(xz_1) = z_2, \quad \dots, \quad f_{\star}(xz_{1:T-1}) = z_T.$$

This richer supervision reduces the dependence to a logarithmic form in their general VC analysis.

4.2 Hanneke–Mehalel–Moran: removing T from sample count

A later result asks whether even the residual $\log T$ dependence is necessary. For finite-VC base classes, an appropriate learner can achieve a CoT sample bound of the representative form

$$m_{\mathcal{F}}^{\text{CoT},T}(\epsilon, \delta) \leq c(\mathcal{F}) \left(\frac{1}{\epsilon} \log \frac{1}{\epsilon} + \log \frac{1}{\delta} \right),$$

where $c(\mathcal{F})$ is independent of generation length T .

Key point A T -independent number of *training trajectories* does not make long traces computationally free. Sample count, token count, and training FLOPs are different resources.

How to say it in the presentation Intermediate supervision can make a long computation statistically easier to identify because one trajectory reveals many applications of the same transition rule. This is a statement about independent examples, not about the cost of processing the trace.

5

How informative is the intermediate trace?

5.1 CoT information

For two hypotheses h_1, h_2 , let p_{agree} be the probability that they agree both on the CoT trace and the final answer. Define

$$I_D^{\text{CoT}}(h_1, h_2) = -\log p_{\text{agree}}.$$

Relative to a target h_* , define the worst discrimination rate among hypotheses with end-to-end error at least ϵ :

$$I_{D, h_*}^{\text{CoT}}(\epsilon; \mathcal{H}) = \inf_{\text{Re}^{2e}(h) \geq \epsilon} I_D^{\text{CoT}}(h, h_*).$$

For a finite class, a union bound gives the characteristic sample scale

$$m \gtrsim \frac{\log |\mathcal{H}| + \log(1/\delta)}{I_{D, h_*}^{\text{CoT}}(\epsilon; \mathcal{H})}.$$

Larger CoT information means incorrect hypotheses are exposed more quickly by intermediate traces.

Caution CoT information measures statistical distinguishability. It is not a measure of readability, factuality, or causal faithfulness of an explanation.

How to say it in the presentation A useful intermediate trace is one that eliminates wrong hypotheses quickly. That statistical usefulness should not be confused with the trace being an honest verbal report of the model's internal causal computation.

6

Reasoning-token lower bounds: BAPO

6.1 Why a separate lower bound is needed

Sample-complexity theorems count training examples. BAPO-style results instead ask how many reasoning tokens a trained system needs at inference time when each step has bounded information access/bandwidth.

6.2 Representative lower bounds

In constant-bandwidth BAPO abstractions, representative tasks exhibit lower bounds such as

$$\text{Majority} : \Omega(n), \quad \text{Match3} : \Omega(n),$$

and graph reachability can require a number of rounds linear in the relevant edge-count parameter.

The proof pattern is a fooling-pair argument: construct inputs x, x' with different answers but arranged so that, if the reasoning budget R is too small, every bounded-information reasoning step has the same observable state on both inputs. Inductively,

$$z_t(x) = z_t(x') \quad (t \leq R),$$

forcing an incorrect identical final answer.

Caution A BAPO lower bound is a theorem for a bounded-information-flow abstraction, not a universal lower bound for every real GPT-style architecture.

6.3 Why this does not contradict sample-complexity gains

The two quantities are different:

$$\underbrace{m}_{\text{training trajectories}} \quad \text{versus} \quad \underbrace{R}_{\text{inference reasoning tokens}}.$$

A rule may be identifiable from few supervised trajectories and still require many communication rounds to execute on a new instance.

How to say it in the presentation Not all long reasoning is useless verbosity. Under bounded information flow, some problems provably require many reasoning rounds. Learning the rule with few examples and executing the rule with few tokens are separate theoretical questions.

7

Reasoning length, error accumulation, and faithfulness

7.1 More tokens are not automatically better

A lower bound says that some minimum reasoning length may be necessary. It does not say that arbitrarily extending a trace improves performance. If each local step has error probability p and errors were independent, a toy success model would give

$$P(\text{all } T \text{ steps correct}) = (1 - p)^T,$$

which decreases with T . Real errors are dependent, but the toy model illustrates why unnecessary steps can create more failure opportunities.

7.2 Faithfulness is an orthogonal axis

A trace can be computationally useful or statistically informative without faithfully describing the hidden mechanism. In particular:

- an optimizer-equivalent circuit can exist without producing a verbal explanation of that circuit;
- a CoT trace can discriminate hypotheses without being causally necessary for the final answer;
- a model can generate plausible reasoning after the decisive internal computation has already occurred.

Mechanistic faithfulness requires causal tests, interventions, or architecture-specific analysis beyond the expressivity/sample-complexity theorems.

How to say it in the presentation “The model learns in context” can mean at least three things: an optimizer-equivalent function is representable, training actually finds that circuit, or the natural-language trace faithfully reports the circuit. Only the first follows directly from the strongest construction theorems discussed here.

8

Putting the four axes in one table

Axis	Main resource	Typical result	What it does not imply
CoT computational depth	decoding steps T	$\text{TIME}(t) \subseteq \text{CoT}(t)$ under assumptions	faithful explanation
ICL $\leftrightarrow$ optimization	layer depth / attention primitive	1-step GD to $(2N + 4)L$ backprop simulation	SGD discovers the circuit
CoT supervision	trajectories m	reduced or removed T dependence in sample count	cheap inference
Token lower bounds	reasoning budget R	linear lower bounds for BAPO-hard tasks	universal LLM lower bound

8.1 Five formulas worth keeping

$$\text{TIME}(t(n)) \subseteq \text{CoT}(t(n)),$$

$$\Delta W = -\frac{\eta}{N} \sum_i (Wx_i - y_i)x_i^\top,$$

$$L \text{ GD steps on } N\text{-layer net} \iff (2N + 4)L \text{ Transformer layers,}$$

$$m_{\mathcal{F}}^{\text{CoT}, T}(\epsilon, \delta) \text{ can be independent of } T,$$

$$R_{\text{reason}} = \Omega(n) \text{ for representative BAPO-hard tasks.}$$

8.2 Common mistakes

- Treating an existence construction as a theorem about what outer-loop SGD learns.
- Treating encoder versus decoder as the main distinction while ignoring the attention primitive and depth of the simulated optimizer.
- Confusing training sample complexity with inference token complexity.
- Equating informative intermediate supervision with causal faithfulness.
- Applying a lower bound proved for BAPO directly to all practical LLMs.

How to say it in the presentation Thirty-second conclusion: CoT can add sequential computation, while a single forward pass can itself contain an optimizer ranging from one GD step to a simulated deep backprop loop. Intermediate supervision can reduce the number of training trajectories needed, yet some bounded-information models still require many inference tokens. These are four different resources and four different theorem families.

Bibliography

- Merrill, W. and Sabharwal, A. *The Expressive Power of Transformers with Chain of Thought*. 2024.
- von Oswald, J. et al. *Transformers Learn In-Context by Gradient Descent*. arXiv:2212.07677.
- Akyürek, E. et al. *What Learning Algorithm Is In-Context Learning? Investigations with Linear Models*. arXiv:2211.15661.
- Bai, Y., Chen, F., Wang, H., Xiong, C., and Mei, S. *Transformers as Statisticians: Provable In-Context Learning with In-Context Algorithm Selection*. arXiv:2306.04637.
- Wu, W. et al. *In-Context Deep Learning via Transformer Models*. arXiv:2411.16549.
- Joshi, N. et al. *On the Sample Complexity of Learning with Chain-of-Thought*. 2025.
- Hanneke, S., Mehal, I., and Moran, S. *Work on CoT supervision and T -independent sample complexity*, 2026.
- Altabaa, A., Montasser, O., and Lafferty, J. *CoT Information: Improved Sample Complexity under Chain-of-Thought Supervision*. NeurIPS, 2025.
- Tomlinson, A. et al. *Reasoning about Reasoning: BAPO Bounds on Chain-of-Thought Token Complexity in LLMs*. 2026.